

PROGRAMAÇÃO MODULAR

EXCEÇÕES E TRATAMENTO DE ERROS

PROF. JOÃO CARAM

XULAMBS PIZZA

2

- Cada pizza pedida está associada a um pedido.
- Queremos armazenar os dados em arquivos no final da execução do sistema.

PEDIDOS E PIZZAS

As pizzas devem ser associadas a um pedido.

```
Pedido novoPedido = new PedidoEntrega(dist);  
...  
novoPedido.adicionar(umaPizza);
```

PEDIDOS E PIZZAS

As pizzas devem ser associadas a um pedido.

A distância é válida?

```
Pedido novoPedido = new PedidoEntrega(dist);
```

...

```
novoPedido.adicionar(umaPizza);
```

É possível adicionar?

A Pizza foi criada?

PEDIDOS E PIZZAS

Os pedidos devem ser salvos em arquivo.

```
File arq = new File("dadosPedidos.bin");  
FileOutputStream fos = new ...  
  
listaPedidos.salvar(fos);
```

PEDIDOS E PIZZAS

Os pedidos devem ser salvos em arquivo.

```
File arq = new File("dadosPedidos.bin");  
FileOutputStream fos = new ...  
  
listaPedidos.salvar(fos);
```

Arquivos existem?

Usuário tem permissão para salvar?

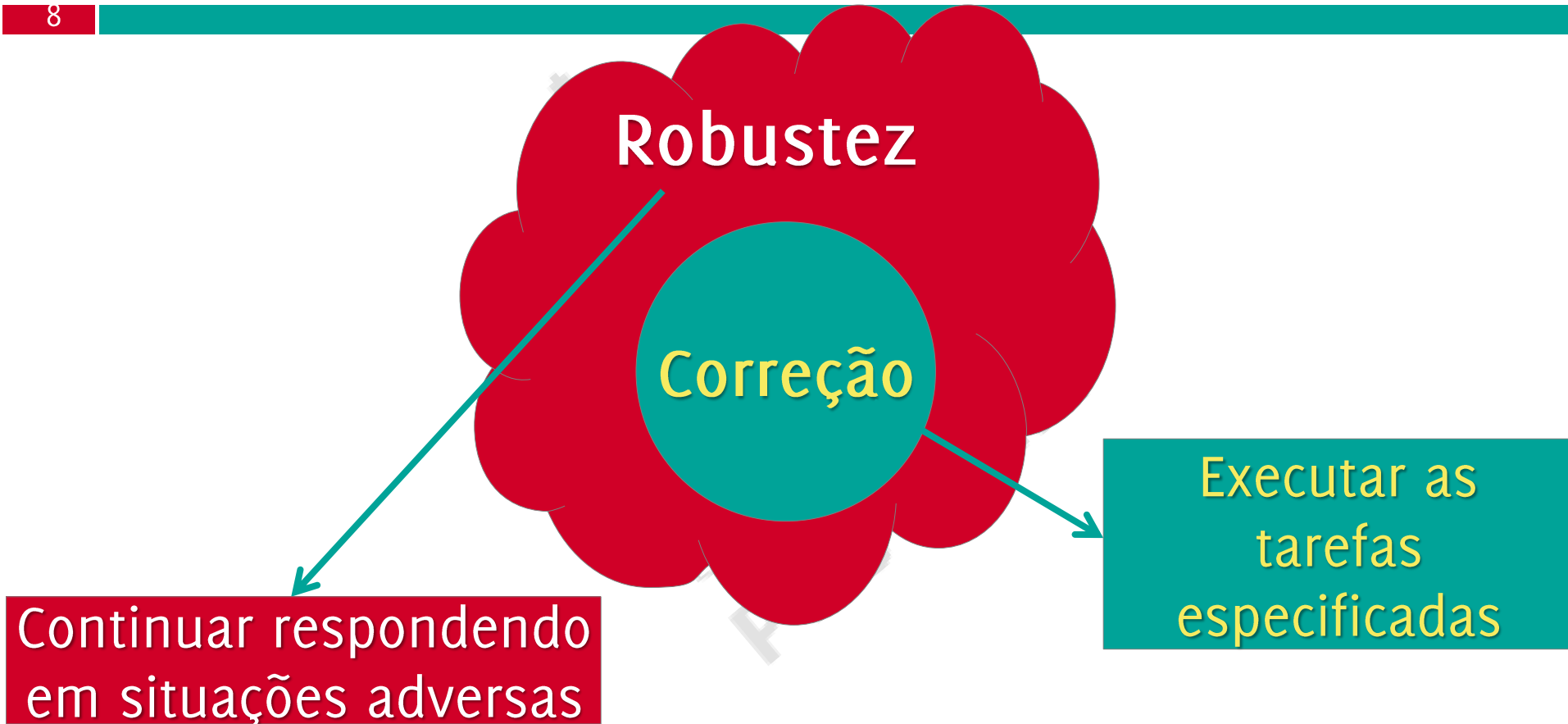
PROGRAMAÇÃO DEFENSIVA

7

- Conceito baseado na noção de direção defensiva:
“Se alguém fizer algo perigoso, você está preparado para evitar consequências. (...) Você assume a responsabilidade pela sua saúde, mesmo que o erro seja do outro motorista.”
- Ideia principal na programação:
 - Problemas inevitavelmente acontecerão. Seu código estará preparado para lidar com eles.

ROBUSTEZ E CORREÇÃO

8

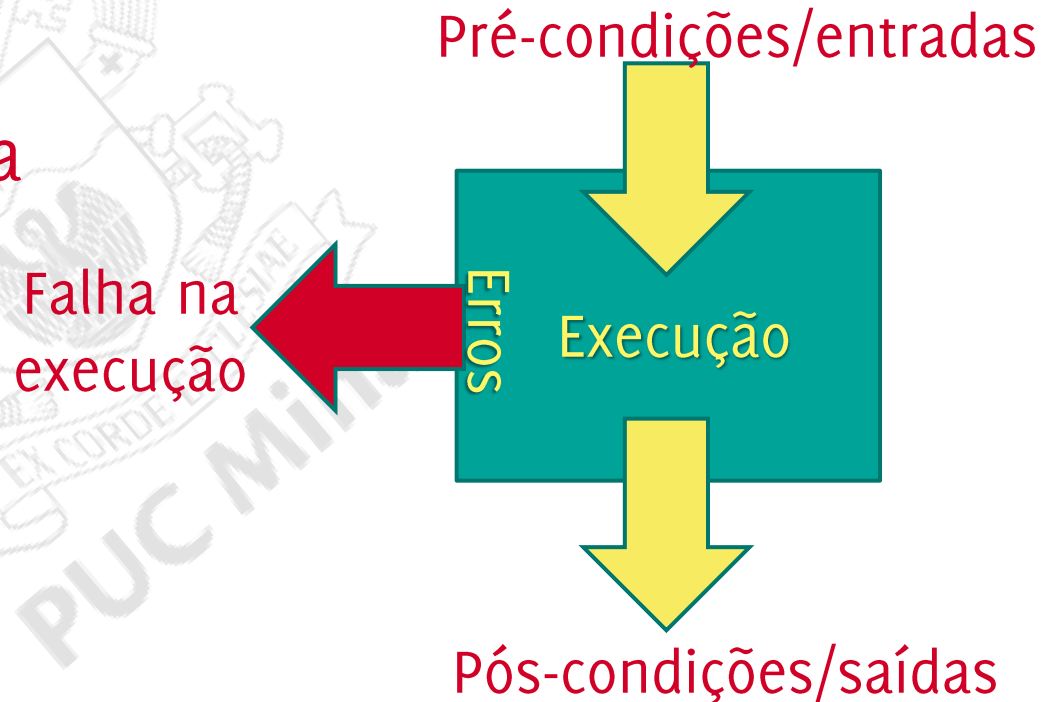


PROGRAMAÇÃO POR CONTRATO

9

- Interfaces de componentes de software devem ser especificadas de maneira formal, precisa e verificável.

- Pré-condições.
- Pós-condições.
- Invariantes.



PROGRAMAÇÃO POR CONTRATO

10

- Método chamado (pré-condições):
 - ou executa (pós-condições),
 - ou falha.
- Falha: situação excepcional.
 - Tratamento varia com o tipo de erro.

PROGRAMAÇÃO POR CONTRATO

11

- ▣ Tratamento formal de falhas:
 - ▣ Separa tratamento do erro do código esperado.
 - ▣ Propaga erros na pilha de chamada de funções.
 - ▣ Agrupa e diferencia tipos de erros.
 - ▣ Java: *Exception* ou *Error*.

12

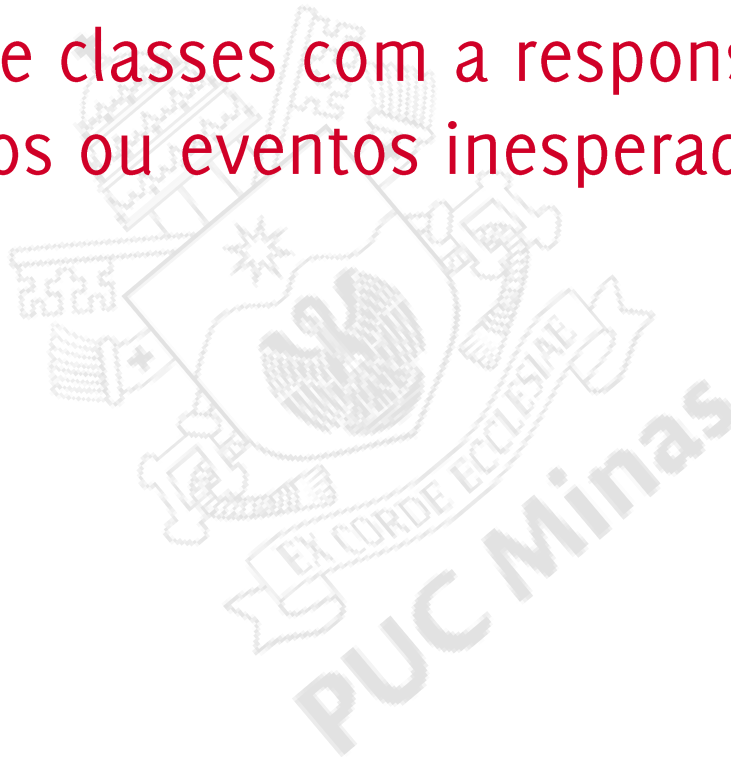
EXCEÇÕES

CAPTURAR, CRIAR, LANÇAR, PROPAGAR...

EXCEÇÕES

13

- Hierarquia de classes com a responsabilidade de sinalizar erros ou eventos inesperados a um código.



EXCEÇÕES

14

- Hierarquia de classes com a responsabilidade de sinalizar erros ou eventos inesperados a um código.

ROBUSTEZ

EXCEÇÕES

15

- Ajudam a impedir que condições de erro sejam propagadas pelo código:
 - Não são ignoráveis.
- Podem reduzir a complexidade do código e melhorar a legibilidade:
 - Centralizam o tratamento de problemas.

EXCEÇÕES

16

- Ajudam a impedir que condições de erro sejam propagadas pelo código:
 - Não são ignoráveis.
- Podem reduzir a complexidade do código e melhorar a legibilidade:
 - Centralizam o tratamento de problemas.
- **MAS DEVEM SER USADAS RESPONSAVELMENTE!!**

EXCEÇÕES E JAVA

```
try{  
    . . .           //bloco de código padrão  
}  
catch (exceção e1){  
    . . .           //blocos de tratamento de erros  
}  
catch (exceção e2){  
    . . .  
}  
finally {           //algumas linguagens oferecem finally  
    . . .           //bloco que será sempre executado  
}
```

LEITURA DE ARQUIVO COM CÓDIGO DE ERRO

```
int LerDados{
    int codigoErro = 0;
    if (AbrirArquivo()) {
        if (ObterTamanhoArquivo()!=-1) {
            if (AlocarMemoria()!=null) {
                LerArquivoNaMemoria();
                if(ErroDeLeitura())
                    codigoErro = -1;
            }else codigoErro = -2;
        }else codigoErro = -3;
        if (!FecharArquivo() && codigoErro == 0)
            codigoErro = -4;
        else
            codigoErro = codigoErro-5;
    }
    else codigoErro = -5;
    return codigoErro ;
}
```

LEITURA DE ARQUIVO COM EXCEÇÕES

```
LeArquivo {  
    try {  
        abrirArquivo();  
        obterTamanhoArquivo();  
        alocarMemoria();  
        lerArquivoNaMemoria();  
        fecharArquivo();  
    } catch ( FalhaAbrirArquivo | FalhaFecharArquivo ) {  
        facaAlgumaCoisa();  
    } catch ( FalhaObtencaoTamanhoArquivo | FalhaLeituraArquivo ) {  
        facaOutraCoisa();  
    } catch ( FalhaAlocacaoMemoria ) {  
        facaAlgoParaResolver();  
    } finally{  
        liberarRecursos();  
    }  
}
```

QUEM DEVE SE PREOCUPAR COM ISSO?

20

- Classes devem avisar caso suas regras estão sendo descumpridas ou recebam dados errados.
- Programa principal deve detectar e tratar todas as anomalias de execução.

ENTENDENDO EXCEÇÕES

21

1. Exceção é lançada.
 - ▣ Ambiente de execução procura um *catch* para tratamento desta exceção.
2. Método em execução não tem este *catch*.
 - ▣ Procura-se o tratamento no método que chamou o método atual... e assim sucessivamente
3. Não é encontrado nenhum bloco *catch*.
 - ▣ Mensagem de exceção sem tratamento para usuário. Interrompe-se a execução do programa.

EXCEÇÕES

22

- Um bloco try pode ter vários blocos catch, cada um lidando com uma exceção diferente.
- Os catch são processados em ordem. É importante escrever primeiro os blocos mais específicos.

EXCEÇÕES E JAVA

23

- Exceções são classes.
 - Ocorrências de exceções são objetos.
- Classes de exceções estão em uma hierarquia.
- Compilador verifica as Checked Exceptions.

O PROGRAMADOR E AS EXCEÇÕES

24

- Usando classes de outros, o programador tem duas alternativas:
 - Capturar e tratar uma exceção que receber (*try/catch*).
 - Propagar a exceção (depois de pensar pelo menos duas vezes sobre isso).

A large, faint watermark of the PUC Minas logo is visible in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

O PROGRAMADOR E AS EXCEÇÕES

26

- Criando suas classes, o programador tem duas alternativas:
 - Lançar uma exceção se houve algo inesperado (*throw*).
 - Exceções da linguagem e exceções próprias.
 - Tratar a exceção recebida de outra classe.

A large, faint watermark of the PUC Minas logo is visible in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

28

BÔNUS 1: TRATAMENTO DE ERROS

SE ALGO DEU ERRADO, O QUE FAZEMOS?

ACONTECEU O ERRO... E ENTÃO?

29

- Apenas capturar a exceção não é o bastante: tratamento adequado deve ser dado ao problema
- Evite propagar exceções se o tratamento pode (deve!) ser local.

RETORNO DE VALOR NEUTRO

30

- Ao acontecer o problema, retorne um valor neutro, válido, que não cause propagação de erro.
- Ex:
 - string vazia
 - Inteiro de valor 0
 - Ponteiro para elemento *null*

SUBSTITUIÇÃO PELO PRÓXIMO VALOR VÁLIDO

31

- Pule para a próxima ocorrência.
- Ex: playlist de áudio, vídeo ou imagens.
várias leituras de um dispositivo/sensor.
geração de listas e outros conjuntos.

RETORNAR A ÚLTIMA RESPOSTA VÁLIDA

32

- Repetir a última resposta válida ou ignorar a tentativa atual, mantendo o estado do objeto.
- Ex:
 - escolha de um canal inexistente na TV/controlador.
 - usar a mesma textura anterior, caso falte uma textura (por exemplo, uma cena 3D).
 - manter uma nota em caso de alteração inválida.

ATRIBUIÇÃO DE VALOR VÁLIDO PRÓXIMO

33

- Calcular um valor válido próximo e atribuir ao objeto, mantendo sua consistência.
- Ex: tentar acertar o relógio para 22h75 (22h59).
 mudar para um canal não existente (mudar
 para o primeiro ou para o último).
 lançar nota maior que a máxima.

USO DE CÓDIGOS DE ERRO

34

- Método usual quando não há exceções.
- Problema principal: *semântica* do código.
 - Ex: posição -1 ou exceção de *objeto não encontrado* em uma busca numa coleção?
- Uso de uma variável de estado e retorno do código de erro no método.

REGISTRO DE ERRO EM *LOGS*

35

- Caso seja uma condição não tratável:
 - Capturar e tratar a exceção (robustez!).
 - Gerar uma mensagem de *log* com a ocorrência.
- Evitar:
 - *Log* de “qualquer coisa”.
 - Gravar no *log* e propagar a exceção.

36

BÔNUS 2: PROJETO E DESEMPENHO



ASPECTOS DE DESEMPENHO

37

- Tratamento de exceções é demorado.
 - se um erro pode ser processado localmente, trate-o, ao invés de lançar uma exceção.
- Propagações de exceções devem ser evitadas em casos esperados: fim de arquivo, por exemplo.
- Exceções são úteis quando dados de entrada não podem ser completamente verificados.

ASPECTOS DE DESEMPENHO

38

- Exceções são úteis:
 - quando não se há o que fazer quando acontece um problema: *se o arquivo não existe, o que fazer?*
 - quando se quer alertar o usuário do objeto ou método.
- Boa prática de programação: se seu método é capaz de tratar uma exceção, então trate-a. Não repasse a responsabilidade.

ASPECTOS DE PROJETO

39

- Planeje antecipadamente a abordagem para o tratamento de erros e uso de exceções.
- Conheça bem as exceções da linguagem utilizada.
- Prefira exceções mais específicas do que genéricas.

ASPECTOS DE PROJETO

40

- Crie exceções que tragam algo adicional, ou use as exceções da linguagem.
- Siga a nomeação padrão.

ASPECTOS DE PROJETO

41

- Utilize o construtor para descrever claramente o problema ocorrido.
 - Mensagens de erro descritivas.
 - Java e parâmetro *cause*.
 - Códigos de erro (ou melhor: enumeráveis)

OBRIGADO.

DÚVIDAS?

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE